

A1:PROPOSAL OF QUESTION

Can we predict the Monthly Charges that we shall charge our customers using Random Forest Regressor?

A2: DEFINED GOAL

The main goal of the data analysis is to predict the monthly charges the telecommunication industry can charge the customers by building a predictive model using a Random Forest Regressor method. Random Forests is an ensemble method that uses a decision tree as a base estimator. The key features in the churn dataset will be analyzed using the Random Forest Regressor and utilized to predict the monthly charge and make recommendations based on the Model's results.

B1:EXPLANATION OF PREDICTION

The regression method chosen to predict the Monthly Charges is the Random Forest Regressor. According to Breiman, Leo (2001, October), Random Forest is an ensemble learning method that combines multiple decision trees to improve predictive accuracy. The random forest regressor builds a collection of decision trees and aggregates their predictions to obtain a final output.

The Random Forest Regressor is an ensemble method that creates a forest of decision trees, where each tree is trained on a different subset of the data. Predictions from all trees are averaged to make the final prediction. Using bootstrap aggregation (bagging), each tree is trained on a randomly selected subset of the training data with replacement, reducing overfitting by ensuring diversity among the trees. A random subset of features is considered at each split, further

reducing the correlation between trees and enhancing overall performance. This approach effectively smoothens predictions, capturing non-linear relationships between features and the target variable while mitigating the risk of overfitting compared to a single decision tree.

In the case of the Churn dataset for the telecommunication industry, the Random Forest Regressor will analyze the data by creating multiple decision trees and making predictions about the monthly charges a customer will incur. Each tree in the forest is built using a random subset of data points and features and will look at different subsets of data and features, during training, the model will split the dataset into subsets and use different attributes to create decision rules in each tree. The final predicted value for Monthly Charges will be derived by averaging the results from all the decision trees in the forest. This ensemble method helps to reduce overfitting and increases the robustness of the random forest regressor model.

The expected outcome from the Random Forest Regressor analysis will be to create a model that can accurately predict the Monthly Charges that the telecommunication industry will charge the customers. The model will identify patterns that influence the charges by analyzing the relationships between the selected features, and from the results, it will be possible to predict the Monthly charges.

B2: SUMMARY OF METHOD ASSUMPTION

According to Charles, Chibuike (2023, June 2), the Random Forest Regressor Method assumes that the predictions from the individual trees must have very low correlations.

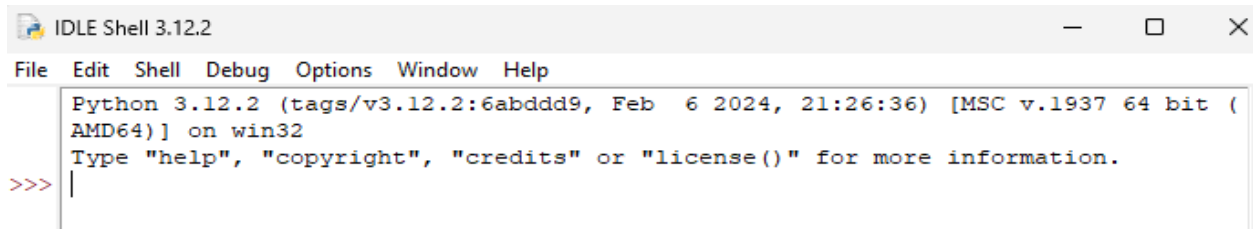
The Random Forest regressor method combines the predictions of multiple decision trees to improve the accuracy and robustness of the model. For this to work well, the trees need to be diverse, which means that their predictions should not be highly correlated with each other

because if the predictions from the individual trees are highly correlated, they would essentially be making similar errors. The benefits of combining the trees would be reduced.

B3: PACKAGES OR LIBRARIES LIST

I will use the Python tool to undertake the analysis using the Random Forest regressor Machine Learning method. Python is an open-source general-purpose programming language with many library packages that can be used and shared with others. I utilized Python Programming Language because of its flexibility and extensive library collections, which are valuable for performing complex calculations and analysis.

For Python, I will be using Python version 3.12.2

A screenshot of the IDLE Shell 3.12.2 window. The window has a title bar with the text 'IDLE Shell 3.12.2' and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the Python 3.12.2 startup message: 'Python 3.12.2 (tags/v3.12.2:6abddd9, Feb 6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32'. Below this message, it says 'Type "help", "copyright", "credits" or "license()" for more information.' and the prompt '>>>' is visible on the left side of the text area.

```
IDLE Shell 3.12.2
File Edit Shell Debug Options Window Help
Python 3.12.2 (tags/v3.12.2:6abddd9, Feb 6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

The following Python libraries will be utilized to perform the various functionalities:

1. Pandas

Pandas will import the data from the CSV file, conduct exploratory data analysis, manipulate data, and convert categorical data to numerical data.

2. Numpy

Numpy will be used for data manipulation.

3. Matplotlib.pyplot

Matplotlib will be used for data visualization.

4. Seaborn

Seaborn will be used for data visualization in the exploratory data analysis, such as creating heatmaps.

5. `from sklearn.model_selection import train_test_split`

The function `train_test_split` from `sklearn.model_selection` will split the dataset into training and testing subsets.

6. `from sklearn import preprocessing`

The library `preprocessing` from `sklearn` will be used for data preparation tasks, such as scaling, encoding categorical variables, and normalizing data.

7. `from sklearn.feature_selection import SelectKBest`

`SelectKBest` from `sklearn.feature_selection` will select the top k features from the dataset based on statistical tests, reducing dimensionality and improving the model.

8. `from sklearn.feature_selection import f_regression`

The function `f_regression` from `sklearn.feature_selection` will be used to perform the ANOVA F-test to select statistically significant features for regression tasks.

9. `from sklearn.ensemble import RandomForestRegressor`

The library `RandomForestRegressor` from `sklearn.ensemble` will be used to implement the Random Forest algorithm for regression tasks

10. `from sklearn.model_selection import GridSearchCV`

The function `GridSearchCV` from `sklearn.model_selection` will perform hyperparameter tuning by testing different combinations of parameters and selecting the best ones for the Random Forest model.

11. `from sklearn.metrics import mean_squared_error`

The function `mean_squared_error` from `sklearn.metrics` will be used to evaluate the performance of the regression model by measuring the average squared difference between predicted and actual values.

12. `from sklearn.metrics import r2_score`

The function `r2_score` from `sklearn.metrics` will be used to evaluate the proportion of variance in the target variable explained by the Random Forest Regressor.

13. `from sklearn.metrics import mean_absolute_error`

The `mean_absolute_error` function from `sklearn.metrics` will be used to calculate the Mean Absolute Error (MAE) between predicted and actual values in a regression model.

14. `warnings`

Additional packages will be used to suppress future warnings and package changes expected from the library to ensure a clean output during the analysis.

```
# 1. Import libraries.
```

```
!pip install pandas
!pip install numpy
!pip install seaborn
!pip install matplotlib
!pip install --upgrade seaborn

#Data Manipulation / Handling data
import pandas as pd #For loading data, data exploration
import numpy as np # for data manipulation

#Data visualization
import matplotlib.pyplot as plt #For plotting feature importance
import seaborn as sns #For Exploratory data visualization to create correlation heatmaps

#Machine Learning
#Data Preprocessing
from sklearn.model_selection import train_test_split # for splitting data
from sklearn import preprocessing #forscaling and encoding

#Feature Selection
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_regression

#Modeling
from sklearn.ensemble import RandomForestRegressor

#Hyperparameter tuning
from sklearn.model_selection import GridSearchCV

#Model Evaluation
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error

#Others
import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore", FutureWarning)
```

C1: DATA PREPROCESSING

1. One data preprocessing goal is to convert categorical data to numerical data through dummy encoding (One Hot encoding).

In the Random Forest Regressor method, preprocessing categorical data through dummy encoding (One Hot Encoding) is crucial to ensure the model can process the data effectively,

as it only accepts numerical inputs. According to Saxena, Shipra (2024, September 18), preprocessing categorical data is necessary since most machine learning models only accept numerical data. One way to do this is through one hot encoding, which is used when the features are nominal, and each category is mapped with a binary variable 0 or 1. 0 represents absence, and 1 represents the presence of that category. The newly created binary features are known as Dummy variables. Dummy encoding uses N-1 features to represent N labels/categories. In this analysis, the first dummy variable will be dropped for the binary columns, whereas all the non-binary columns will be retained.

C2: DATA SET VARIABLES

The following variables from the churn dataset will be used to perform the analysis to predict the Monthly charge.

1. MonthlyCharge

MonthlyCharge is a numeric variable and will be used as the target or response (dependent) variable.

2. Population

The variable Population is numerical. It will be used as a predicting (independent) variable.

3. Area

The variable Area is categorical. It will be used as a predicting (independent) variable.

4. Children

The variable Children is numerical. It will be used as a predicting (independent) variable.

5. Age

The variable Age is numerical. It will be used as a predicting (independent) variable.

6. Income

The variable Income is numerical. It will be used as a predicting (independent) variable.

7. Marital

The variable Marital is categorical. It will be used as a predicting (independent) variable.

8. Gender

The variable Gender is categorical. It will be used as a predicting (independent) variable.

9. Churn

The variable Churn is categorical. It will be used as a predicting (independent) variable.

10. Outage_sec_perweek

The variable Outage_sec_perweek is numerical. It will be used as a predicting
(independent) variable.

11. Email

The variable Email is numerical. It will be used as a predicting (independent) variable.

12. Contacts

The variable Contacts is numerical. It will be used as a predicting (independent) variable.

13. Yearly_equip_failure

The variable Yearly_equip_failure is numerical. It will be used as a predicting
(independent) variable.

14. Techie

The variable Techie is categorical. It will be used as a predicting (independent) variable.

15. Contract

The variable Contract is categorical. It will be used as a predicting (independent) variable.

16. Port_modem

The variable Port_modem is categorical. It will be used as a predicting (independent) variable.

17. Tablet

The variable Tablet is categorical. It will be used as a predicting (independent) variable.

18. InternetService

The variable InternetService is categorical. It will be used as a predicting (independent) variable.

19. Phone

The variable Phone is categorical. It will be used as a predicting (independent) variable.

20. Multiple

The variable Multiple is categorical. It will be used as a predicting (independent) variable.

21. OnlineSecurity

The variable OnlineSecurity is categorical. It will be used as a predicting (independent) variable.

22. OnlineBackup

The variable OnlineBackup is categorical. It will be used as a predicting (independent) variable.

23. DeviceProtection

The variable DeviceProtection is categorical. It will be used as a predicting (independent) variable.

24. TechSupport

The variable TechSupport is categorical. It will be used as a predicting (independent) variable.

25. StreamingTV

The variable StreamingTV is categorical. It will be used as a predicting (independent) variable.

26. StreamingMovies

The variable StreamingMovies is categorical. It will be used as a predicting (independent) variable.

27. PaperlessBilling

The variable PaperlessBilling is categorical. It will be used as a predicting (independent) variable.

28. PaymentMethod

The variable PaymentMethod is categorical. It will be used as a predicting (independent) variable.

29. Tenure

The variable Tenure is numerical. It will be used as a predicting (independent) variable.

30. Bandwidth_GB_Year

The variable Bandwidth_GB_Year is numerical. It will be used as a predicting (independent) variable.

31. Timelyresponse

The variable Timelyresponse is categorical. It will be used as a predicting (independent) variable.

32. Timelyfixes

The variable Timelyfixes is categorical. It will be used as a predicting (independent) variable.

33. Timelyreplacements

The variable Timelyreplacements is categorical. It will be used as a predicting (independent) variable.

34. Reliability

The variable Reliability is categorical. It will be used as a predicting (independent) variable.

35. Options

The variable Options is categorical. It will be used as a predicting (independent) variable.

36. Respectfulresponse

The variable Respectfulresponse is categorical. It will be used as a predicting (independent) variable.

37. Courteousexchange

The variable Courteousexchange is categorical. It will be used as a predicting (independent) variable.

38. Evidenceofactivelistening

The variable Evidenceofactivelistening is categorical. It will be used as a predicting (independent) variable.

C3: STEPS FOR ANALYSIS

In performing the analysis, the following steps were undertaken

1. Loading the necessary libraries and packages that will be utilized for the analysis

```
# 1. Import Libraries.

!pip install pandas
!pip install numpy
!pip install seaborn
!pip install matplotlib
!pip install --upgrade seaborn

#Data Manipulation / Handling data
import pandas as pd #For loading data, data exploration
import numpy as np # for data manipulation

#Data visualization
import matplotlib.pyplot as plt #For plotting feature importance
import seaborn as sns #For Exploratory data visualization to create correlation heatmaps

#Machine Learning
#Data Preprocessing
from sklearn.model_selection import train_test_split # for splitting data
from sklearn import preprocessing #forscaling and encoding

#Feature Selection
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_regression

#Modeling
from sklearn.ensemble import RandomForestRegressor

#Hyperparameter tuning
from sklearn.model_selection import GridSearchCV

#Model Evaluation
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error

#Others
import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore", FutureWarning)
```

2. Loading the data into a dataframe

#can we predict the Monthly charges we shall bill our customers?

#File path variables

```
Local_path = 'C:/Users/beatr/OneDrive/Desktop/D209/Task2/'
InputFilePath = Local_path + 'churn_clean.csv'
OutputfilePath = Local_path + 'churn_cleaned_df.csv'
XtrainOutputfilePath = Local_path + 'Xtrain.csv'
XtestOutputfilePath = Local_path + 'Xtest.csv'
ytrainOutputfilePath = Local_path + 'ytrain.csv'
ytestOutputfilePath = Local_path + 'ytest.csv'
TargetVariable = "MonthlyCharge"
```

2. Load the data.

```
#Loading the data
Churn_df = pd.read_csv(InputFilePath, index_col=0)
Churn_df = pd.read_csv(InputFilePath)
```

3. Viewing the data

#Initial data exploration (Understanding the data)

```
pd.set_option('display.max_columns', None)
Churn_df.head()
```

	CaseOrder	Customer_id	Interaction	UID	City	State	County	Zip	Lat	Lng	Population	Are
0	1	K409198	aa90260b-4141-4a24-8e36-b04ce1f4f77b	e885b299883d4f9fb18e39c75155d990	Point Baker	AK	Prince of Wales-Hyder	99927	56.25100	-133.37571	38	Urba
1	2	S120509	fb76459f-c047-4a9d-8af9-e0f7d4ac2524	f2de8bef964785f41a2959829830fb8a	West Branch	MI	Ogemaw	48661	44.32893	-84.24080	10446	Urba
2	3	K191035	344d114c-3736-4be5-98f7-c72c281e2d35	f1784cfa9f6d92ae816197eb175d3c71	Yamhill	OR	Yamhill	97148	45.35589	-123.24657	3735	Urba
3	4	D90850	abfa2b40-2d43-4894-b15a-989b8c79e311	dc8a365077241bb5cd5ccd305136b05e	Del Mar	CA	San Diego	92014	32.96687	-117.24798	13863	Suburba
4	5	K662701	68a861fd-0d20-4e51-a587-8a90407ee574	aabb64a116e83fdc4befc1fbab1663f9	Needville	TX	Fort Bend	77461	29.38012	-95.80673	11352	Suburba

Churn_df.shape

(10000, 50)

4. Evaluating the data and data structure

```
Churn_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
 #   Column                                  Non-Null Count  Dtype
---  -
 0   CaseOrder                             10000 non-null  int64
 1   Customer_id                           10000 non-null  object
 2   Interaction                            10000 non-null  object
 3   UID                                    10000 non-null  object
 4   City                                   10000 non-null  object
 5   State                                  10000 non-null  object
 6   County                                 10000 non-null  object
 7   Zip                                    10000 non-null  int64
 8   Lat                                    10000 non-null  float64
 9   Lng                                    10000 non-null  float64
10   Population                             10000 non-null  int64
11   Area                                    10000 non-null  object
12   TimeZone                               10000 non-null  object
13   Job                                     10000 non-null  object
14   Children                               10000 non-null  int64
15   Age                                     10000 non-null  int64
16   Income                                 10000 non-null  float64
17   Marital                                10000 non-null  object
18   Gender                                 10000 non-null  object
19   Churn                                  10000 non-null  object
20   Outage_sec_perweek                     10000 non-null  float64
21   Email                                  10000 non-null  int64
22   Contacts                               10000 non-null  int64
23   Yearly equip_failure                   10000 non-null  int64
24   Techie                                 10000 non-null  object
25   Contract                               10000 non-null  object
26   Port_modem                             10000 non-null  object
27   Tablet                                 10000 non-null  object
28   InternetService                       7871 non-null   object
29   Phone                                  10000 non-null  object
30   Multiple                               10000 non-null  object
31   OnlineSecurity                         10000 non-null  object
32   OnlineBackup                           10000 non-null  object
33   DeviceProtection                       10000 non-null  object
34   TechSupport                            10000 non-null  object
35   StreamingTV                            10000 non-null  object
36   StreamingMovies                        10000 non-null  object
37   PaperlessBilling                       10000 non-null  object
38   PaymentMethod                          10000 non-null  object
39   Tenure                                 10000 non-null  float64
40   MonthlyCharge                          10000 non-null  float64
41   Bandwidth_GB_Year                     10000 non-null  float64
42   Item1                                  10000 non-null  int64
43   Item2                                  10000 non-null  int64
44   Item3                                  10000 non-null  int64
45   Item4                                  10000 non-null  int64
46   Item5                                  10000 non-null  int64
47   Item6                                  10000 non-null  int64
48   Item7                                  10000 non-null  int64
49   Item8                                  10000 non-null  int64
dtypes: float64(7), int64(16), object(27)
memory usage: 3.8+ MB
```

5. Data Cleaning and preprocessing

This includes

i. Checking for Nulls and Missing data.

```
: #Checking for Nulls and missing data
# print(Churn_df.isna().sum().sort_values())
print(Churn_df[Churn_df[Churn_df.columns[Churn_df.isna().any()]].columns].info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 1 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InternetService  7871 non-null   object
dtypes: object(1)
memory usage: 78.3+ KB
None

: #Cleaning the data
# Replace NaN with "Other" in the 'InternetServices' column
Churn_df['InternetService'] = Churn_df['InternetService'].fillna('Other')

: #Checking for Nulls and missing data

print(Churn_df[Churn_df[Churn_df.columns[Churn_df.isna().any()]].columns].info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Empty DataFrame
None
```

ii. Cleaning white spaces

```
#Trim and Clean White Spaces
Churn_df = Churn_df.map(lambda x: x.strip() if isinstance(x, str) else x)
```

iii. Cleaning data type constraint

```
#Converting the data types for Categorical data which was loaded as intergers
categorical_columns = ['Item1', 'Item2', 'Item3', 'Item4', 'Item5', 'Item6', 'Item7', 'Item8']
for col in categorical_columns:
    Churn_df[col] = Churn_df[col].astype('object')
print(Churn_df.dtypes)
```

CaseOrder	int64
Customer_id	object
Interaction	object
UID	object
City	object
State	object
County	object
Zip	int64
Lat	float64
Lng	float64
Population	int64
Area	object
TimeZone	object
Job	object
Children	int64
Age	int64
Income	float64
Marital	object
Gender	object
Churn	object
Outage_sec_perweek	float64
Email	int64
Contacts	int64
Yearly equip_failure	int64
Techie	object
Contract	object
Port_modem	object
Tablet	object
InternetService	object
Phone	object
Multiple	object
OnlineSecurity	object
OnlineBackup	object
DeviceProtection	object
TechSupport	object
StreamingTV	object
StreamingMovies	object
PaperlessBilling	object
PaymentMethod	object
Tenure	float64
MonthlyCharge	float64
Bandwidth_GB_Year	float64
Item1	object
Item2	object
Item3	object
Item4	object
Item5	object
Item6	object
Item7	object
Item8	object

dtype: object

iv. Renaming columns for better descriptions

```
# Rename last 8 survey columns for better description of variables
Churn_df.rename(columns = {'Item1':'Timelyresponse',
                           'Item2':'Timelyfixes',
                           'Item3':'Timelyreplacements',
                           'Item4':'Reliability',
                           'Item5':'Options',
                           'Item6':'Respectfulresponse',
                           'Item7':'Courteousexchange',
                           'Item8':'Evidenceofactivelistening'},
                 inplace=True)
```

v. Checking for duplicates

```
#Checking for duplicated data

print("Duplicated rows")
print(Churn_df.duplicated().any())
print(" ")
print("Duplicated columns")
print(Churn_df.columns.duplicated().any())
```

Duplicated rows
False

Duplicated columns
False

- vi. Checking the number of unique values per feature (Checking for high cardinality)

```
#Checking for unique values
```

```
Churn_df.nunique()
```

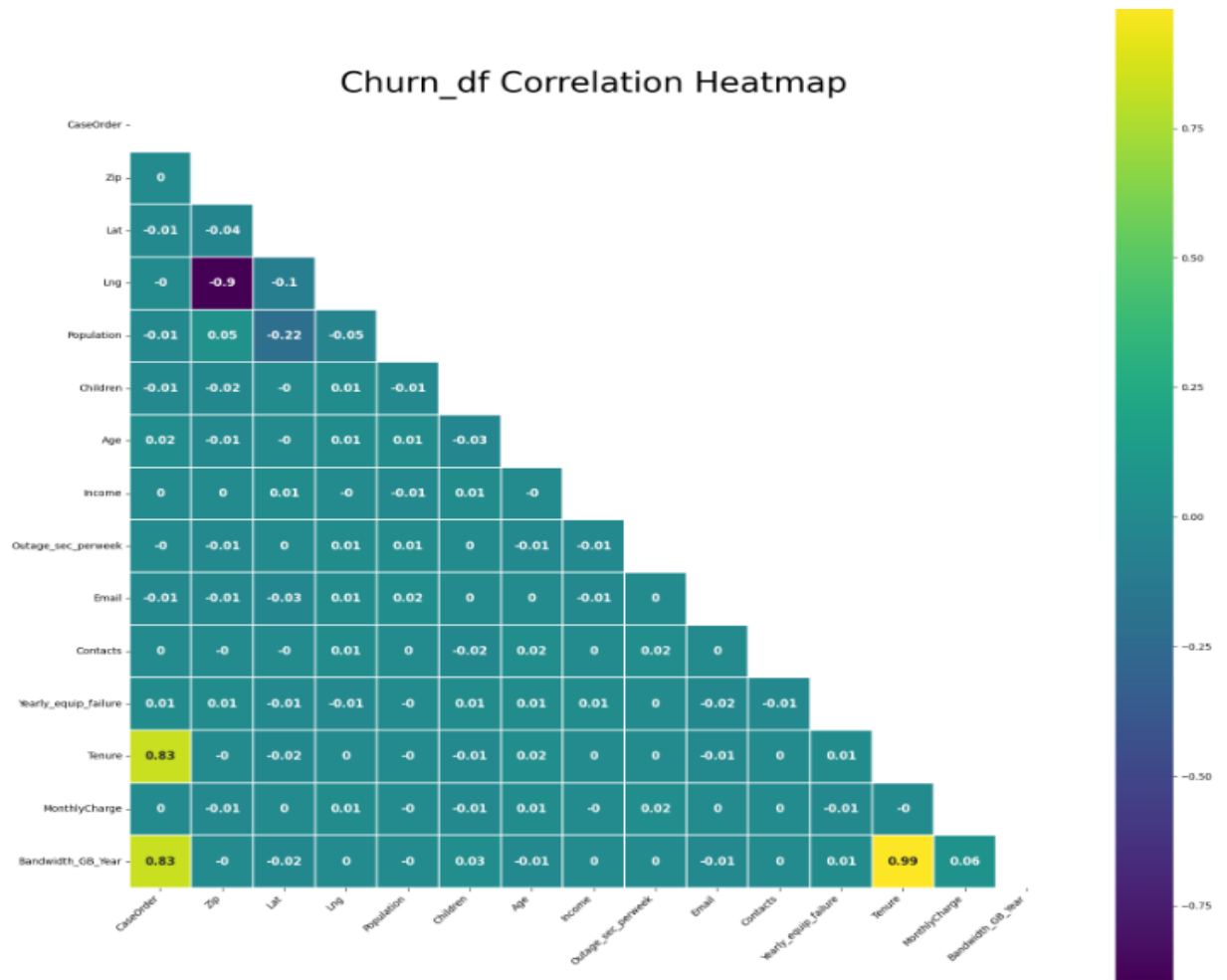
CaseOrder	10000
Customer_id	10000
Interaction	10000
UID	10000
City	6058
State	52
County	1620
Zip	8583
Lat	8563
Lng	8630
Population	5933
Area	3
TimeZone	25
Job	639
Children	11
Age	72
Income	9993
Marital	5
Gender	3
Churn	2
Outage_sec_perweek	9986
Email	23
Contacts	8
Yearly_equip_failure	6
Techie	2
Contract	3
Port_modem	2
Tablet	2
InternetService	3
Phone	2
Multiple	2
OnlineSecurity	2
OnlineBackup	2
DeviceProtection	2
TechSupport	2
StreamingTV	2
StreamingMovies	2
PaperlessBilling	2
PaymentMethod	4
Tenure	9996
MonthlyCharge	750
Bandwidth_GB_Year	10000
Timelyresponse	7
Timelyfixes	7
Timelyreplacements	8
Reliability	7
Options	7
Respectfulresponse	8
Courteousexchange	7
Evidenceofactivelistening	8

dtype: int64

vii. Checking for correlation

```
#Analysing the Correlation /Relationship analysis
Continuous_columns = Churn_df.select_dtypes(include=['float64', 'int64']).columns.tolist()
Cor_data = Churn_df[Continuous_columns]

def Generate_heatmap_graph(corr, chart_title, mask_uppertri=False):
    """ Based on features, generate a correlation matrix """
    mask = np.zeros_like(corr, dtype=bool)
    mask[np.triu_indices_from(mask)] = mask_uppertri
    fig, ax = plt.subplots(figsize=(20, 20))
    sns.heatmap(corr,
                mask=mask,
                square=True,
                annot=True,
                annot_kws={'size': 12, 'rotation': 0, 'weight': 'bold'},
                cmap=plt.get_cmap('viridis'),
                linewidths=.1,
                ax=ax)
    ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right') # Rotate x-axis labels
    plt.title(chart_title, fontsize=32)
    plt.show()
var_corr = round(Cor_data.corr(), 2)
Generate_heatmap_graph(var_corr, chart_title='Churn_df Correlation Heatmap', mask_uppertri=True)
```



viii. Dropping features that will not be used for analysis

```
#Dropping Columns that will not be used in the analysis
Churn_df = Churn_df.drop(columns = ['CaseOrder', 'Customer_id', 'Interaction', 'UID', 'City', 'State', 'County', 'Zip', 'Lat', 'Lng', 'Job', 'TimeZone'])
```

ix. Dummy encoding of categorical variables

Converting Categorical data to numerical data through dummy encoding. Include all dummies for non-binary variables and include only one dummy variable for binary columns.

```
#Converting Categorical data to numerical data through dummy encoding (One Hot encoding)

#Dummy encoding is often used as an alternative term.
#However, dummy encoding typically drops one of the binary columns to prevent multicollinearity (unlike standard one-hot encoding).
#The method uses the option 'drop_first=True'.

categorical_columns = Churn_df.select_dtypes(include=['object']).columns.tolist()

# Separate binary and non-binary categorical columns
binary_columns = [col for col in categorical_columns if Churn_df[col].nunique() == 2]
non_binary_columns = [col for col in categorical_columns if Churn_df[col].nunique() > 2]

# Create dummy variables
Churn_df_encoded = pd.get_dummies(
    Churn_df,
    columns=non_binary_columns, # Include all dummies for non-binary variables
    dummy_na=False,
    drop_first=False, # Do not drop any dummy for non-binary
    dtype='int64'
)

# For binary columns, include only one dummy variable
Churn_df_encoded = pd.get_dummies(
    Churn_df_encoded,
    columns=binary_columns,
    dummy_na=False,
    drop_first=True, # Drop the first dummy for binary
    dtype='int64'
)

# Display the resulting dataframe
Churn_df_encoded.head()
```

```
: # variable for numeric data
numerical_features = Churn_df_encoded.select_dtypes(include="number").columns
print("numerical_features")
for feature in numerical_features:
    print(feature)
```

```
numerical_features
Population
Children
Age
Income
Outage_sec_perweek
Email
Contacts
Yearly_equip_failure
Tenure
MonthlyCharge
Bandwidth_GB_Year
Area_Rural
Area_Suburban
Area_Urban
Marital_Divorced
Marital_Married
Marital_Never Married
Marital_Separated
Marital_Widowed
Gender_Female
Gender_Male
Gender_Nonbinary
Contract_Month-to-month
Contract_One year
Contract_Two Year
InternetService_DSL
InternetService_Fiber Optic
InternetService_Other
PaymentMethod_Bank Transfer(automatic)
PaymentMethod_Credit Card (automatic)
PaymentMethod_Electronic Check
PaymentMethod_Mailed Check
Timelyresponse_1
Timelyresponse_2
Timelyresponse_3
Timelyresponse_4
Timelyresponse_5
Timelyresponse_6
Timelyresponse_7
Timelyfixes_1
Timelyfixes_2
Timelyfixes_3
Timelyfixes_4
Timelyfixes_5
Timelyfixes_6
Timelyfixes_7
Timelyreplacements_1
Timelyreplacements_2
Timelyreplacements_3
Timelyreplacements_4
Timelyreplacements_5
Timelyreplacements_6
Timelyreplacements_7
```

Timelyreplacements_8
Reliability_1
Reliability_2
Reliability_3
Reliability_4
Reliability_5
Reliability_6
Reliability_7
Options_1
Options_2
Options_3
Options_4
Options_5
Options_6
Options_7
Respectfulresponse_1
Respectfulresponse_2
Respectfulresponse_3
Respectfulresponse_4
Respectfulresponse_5
Respectfulresponse_6
Respectfulresponse_7
Respectfulresponse_8
Courteousexchange_1
Courteousexchange_2
Courteousexchange_3
Courteousexchange_4
Courteousexchange_5
Courteousexchange_6
Courteousexchange_7
Evidenceofactivelistening_1
Evidenceofactivelistening_2
Evidenceofactivelistening_3
Evidenceofactivelistening_4
Evidenceofactivelistening_5
Evidenceofactivelistening_6
Evidenceofactivelistening_7
Evidenceofactivelistening_8
Churn_Yes
Techie_Yes
Port_modem_Yes
Tablet_Yes
Phone_Yes
Multiple_Yes
OnlineSecurity_Yes
OnlineBackup_Yes
DeviceProtection_Yes
TechSupport_Yes
StreamingTV_Yes
StreamingMovies_Yes
PaperlessBilling_Yes

x. Scaling features

```
# Separate the X variables (explanatory) from the Y variable (response) into different dataframes
df_X = Churn_df_encoded.drop([TargetVariable], axis=1).copy()
df_y = Churn_df_encoded[TargetVariable].copy()
```

```
# Standardize the X variables (explanatory) to put all on the same scale for Random forest regressor
df_X = pd.DataFrame(preprocessing.MinMaxScaler().fit_transform(df_X), columns=df_X.columns)
df_X.head()
```

	Population	Children	Age	Income	Outage_sec_perweek	Email	Contacts	Yearly equip_failure	Tenure	Bandwidth_GB_Year	Area_Rural	Are
0	0.000340	0.0	0.704225	0.109120	0.373260	0.409091	0.000000	0.166667	0.081624	0.106951	0.0	
1	0.093393	0.1	0.126761	0.082599	0.549537	0.500000	0.000000	0.166667	0.002203	0.092165	0.0	
2	0.033393	0.4	0.450704	0.035818	0.504705	0.363636	0.000000	0.166667	0.207804	0.271180	0.0	
3	0.123943	0.1	0.422535	0.071848	0.701827	0.636364	0.285714	0.000000	0.226580	0.286868	0.0	
4	0.101493	0.0	0.915493	0.153646	0.381271	0.681818	0.285714	0.166667	0.009447	0.016561	0.0	

xi. Feature selection.

Feature selection using SelectKBest involves selecting the top 15 features based on a statistical test to improve model performance and reduce overfitting.


```
# 6. Feature Selection
```

```
#Identify the optimal number of features to select

k =15 # Number of features to select
selector = SelectKBest(score_func=f_regression, k=k)
X_selected = selector.fit_transform(df_X, df_y)

# Get the selected feature names
Selected_Features = df_X.columns[selector.get_support()]
print("Selected features:")
for feature in Selected_Features:
    print(feature)
```

```
Selected features:
Bandwidth_GB_Year
InternetService_DSL
InternetService_Fiber Optic
InternetService_Other
Timelyresponse_3
Options_3
Evidenceofactivelistening_3
Churn_Yes
Multiple_Yes
OnlineSecurity_Yes
OnlineBackup_Yes
DeviceProtection_Yes
TechSupport_Yes
StreamingTV_Yes
StreamingMovies_Yes
```

6. Exporting the cleaned data

7. Export the Cleaned Data

```
# Recombine df_X (feature set) and df_y (response variable) to save the whole thing in one dataframe
Churn_Cleaned_df = df_X[Selected_Features].copy() # Only keep the selected features
Churn_Cleaned_df[TargetVariable] = df_y.reset_index(drop=True)
```

```
# Save dataframe to CSV, ignore ind
```

```
print('The Cleaned Data')
print(f'Churn_Cleaned_df.shape: {Churn_Cleaned_df.shape}')
Churn_Cleaned_df.info()
```

```
The Cleaned Data
Churn_Cleaned_df.shape: (10000, 16)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 16 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Bandwidth_GB_Year                    10000 non-null  float64
 1   InternetService_DSL                  10000 non-null  float64
 2   InternetService_Fiber Optic          10000 non-null  float64
 3   InternetService_Other                 10000 non-null  float64
 4   Timelyresponse_3                     10000 non-null  float64
 5   Options_3                            10000 non-null  float64
 6   Evidenceofactivelistening_3          10000 non-null  float64
 7   Churn_Yes                             10000 non-null  float64
 8   Multiple_Yes                         10000 non-null  float64
 9   OnlineSecurity_Yes                   10000 non-null  float64
10   OnlineBackup_Yes                     10000 non-null  float64
11   DeviceProtection_Yes                 10000 non-null  float64
12   TechSupport_Yes                      10000 non-null  float64
13   StreamingTV_Yes                      10000 non-null  float64
14   StreamingMovies_Yes                  10000 non-null  float64
15   MonthlyCharge                        10000 non-null  float64
dtypes: float64(16)
memory usage: 1.2 MB
```

C4: CLEANED DATA SET

cleaned data set.



churn_cleaned_df.csv

D1:SPLITTING THE DATA

Split the data into training and test data sets using train_test_split.

The data will be split into X_train, X_test, y_train, and y_test to separate training data that will be used to build the model from testing data that will be used to evaluate the model's performance. This will ensure the model is trained and validated on distinct datasets, preventing overfitting and generalizing well to new data. The data will be split into training and testing subsets, with 80% allocated for training and 20% for testing; stratification will ensure that class proportions are maintained across both subsets

```
# 8. Split the Data - split the cleaned data.
```

```
# Split the data into training and test sets (80% training, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_selected, df_y, test_size=0.2, random_state=42)
```

```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(8000, 10)
(2000, 10)
(8000,)
(2000,)
```

```
# Convert X_train, X_test, y_train, and y_test into DataFrames or Series
X_train_df = pd.DataFrame(X_train, columns=Selected_Features)
X_test_df = pd.DataFrame(X_test, columns=Selected_Features)
y_train_df = pd.Series(y_train)
y_test_df = pd.Series(y_test)

# Save each of the split data for Xtrain,Xtest,ytrain,y,test
X_train_df.to_csv(XtrainOutputfilePath, index=False)
X_test_df.to_csv(XtestOutputfilePath, index=False)
y_train_df.to_csv(ytrainOutputfilePath, index=False)
y_test_df.to_csv(ytestOutputfilePath, index=False)
```



Xtrain.csv



Xtest.csv



ytrain.csv



ytest.csv

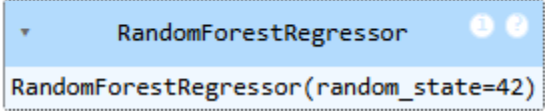
D2:OUTPUT AND INTERMEDIATE CALCULATIONS

An initial model Random Forest regressor model was created with default hyperparameters, and the random_state was set to 42 to ensure reproducibility.

```
: # 9. Create or initialize the model - call the RandomForestRegressor  
:  
: initial_model = RandomForestRegressor(random_state=42)
```

Then, the model was fit into the training data set

```
# 10. Train and fit the model.  
  
initial_model.fit(X_train, y_train)
```



The image shows a Jupyter Notebook cell output. It displays the class name 'RandomForestRegressor' in a blue box, followed by the instance 'RandomForestRegressor(random_state=42)' in a white box. There are also some small icons (a question mark and a refresh symbol) next to the class name.

The Models will be evaluated based on the metrics below

1. Mean Squared Error (MSE)

Mean Squared Error (MSE) is a metric used to evaluate the performance of a regression model. It measures the average squared difference between predicted and actual values. MSE quantifies the overall error of the model in predicting the target variable.

2. R^2 Score

The R^2 score, or the coefficient of determination, is a statistical metric used to evaluate the performance of a regression model. It measures the proportion of the variance in the dependent variable that is predictable from the independent variables.

3. Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) is a metric used to evaluate the accuracy of a predictive model by measuring the average magnitude of the prediction errors. It represents the square root of the Mean Squared Error (MSE)

4. Mean Absolute Error (MAE)

MAE is a regression metric that measures the average magnitude of errors without squaring them, making it less sensitive to outliers.

The initial model was evaluated using the testing dataset.

Mean Squared Error: 0.000498136

R² Score: 0.9999997339

Root Mean Squared Error (RMSE): 0.02231897

Mean Absolute Error (MAE): 0.002659524

The MSE measure of the average squared difference between predicted and actual values was low, indicating better accuracy. MSE of 0.000498136 suggests the model is performing well with minimal errors.

The R² score indicates how well the model explains the variance in the data. Since the value was close to 1, the model explains nearly all the variance in the target variable (MonthlyCharge). The high score implies the model is a near-perfect fit to the data, capturing the relationship between input features and target very well.

RMSE is the square root of MSE and represents the standard deviation of the residuals (prediction errors). An RMSE of 0.02231897 means the model's prediction error, on average, is about 0.0223 units. While not zero, this still indicates a high level of accuracy, with relatively small deviations from the actual values.

MAE is the average of the absolute errors between predicted and actual values. A low MAE of 0.002659524 suggests that, on average, the model's predictions are off by just 0.00266 units.

This shows that the initial model performs very well regarding prediction accuracy.

```
# 11. Evaluate the model for Accuracy and check the error metrics
```

```
y_pred_initial = initial_model.predict(X_test)
mse_initial = mean_squared_error(y_test, y_pred_initial)
r2_initial = r2_score(y_test, y_pred_initial)
print(f"Initial Model - Mean Squared Error: {mse_initial}")
print(f"Initial Model - R^2 Score: {r2_initial}")
rmse_initial = mse_initial ** 0.5
mae_initial = mean_absolute_error(y_test, y_pred_initial)

print(f"Initial Model - Root Mean Squared Error (RMSE): {rmse_initial}")
print(f"Initial Model - Mean Absolute Error (MAE): {mae_initial}")
```

```
Initial Model - Mean Squared Error: 0.0004981363706099721
Initial Model - R^2 Score: 0.9999997338708149
Initial Model - Root Mean Squared Error (RMSE): 0.022318968851852725
Initial Model - Mean Absolute Error (MAE): 0.00265952384883871
```

To improve performance, GridSearchCV was used for systematic hyperparameter tuning, ensuring the model was optimized for performance.

The following parameters were used for optimization

1. `n_estimators`

The `n_estimators` define the number of trees in the ensemble.

2. `max_depth`

The `max_depth` controls the maximum depth of each decision tree in the ensemble.

3. `min_samples_split`

The `min_samples_split` controls the minimum number of samples to split an internal tree node.

4. `min_samples_leaf`

The `min_samples_leaf` defines the minimum number of samples a leaf node must have.

The optimized model, with the best hyperparameters (`n_estimators=200`, `max_depth=None`, `min_samples_leaf=2`, and `min_samples_split=10`), was retrained and tested on the test dataset.

The Final model was evaluated using the testing dataset.

Mean Squared Error: 0.0003010796

R² Score: 0.9999998391

Root Mean Squared Error (RMSE): 0.017351646

Mean Absolute Error (MAE): 0.002372828

The MSE for the final model is lower than that of the initial model, indicating that the model has improved after optimization. This reduction in MSE means the final model makes more accurate predictions with a smaller average squared error.

The R² score has slightly increased compared to the initial model, reaching 0.9999998391. This means the final model explains more of the data's variance than the initial model, reflecting a better fit.

The RMSE has decreased from 0.02231897 to 0.017351646, which shows an improvement in the model's accuracy. The error per prediction has reduced, indicating that the final model's predictions are closer to the actual values than the initial model.

The MAE has decreased slightly compared to the initial model (from 0.002659524 to 0.002372828), suggesting that the final model has made more accurate predictions on average, showing a better performance.

After optimization, the final model performs slightly better across all metrics, confirming that hyperparameter tuning (using GridSearchCV) improved the model's predictive accuracy.

```
# 12. Optimize the model (includes hyperparameter tuning or cross-validation).
```

```
# Optimize the model with GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
grid_search = GridSearchCV(estimator=RandomForestRegressor(random_state=42),
                           param_grid=param_grid,
                           cv=3,
                           scoring='r2',
                           verbose=2,
                           n_jobs=-1)
grid_search.fit(X_train, y_train)
best_params = grid_search.best_params_
print("Best Parameters from GridSearchCV:", best_params)

final_model = grid_search.best_estimator_

# 6. Evaluate the final model
y_pred_final = final_model.predict(X_test)
mse_final = mean_squared_error(y_test, y_pred_final)
r2_final = r2_score(y_test, y_pred_final)
print(f"Final Model - Mean Squared Error: {mse_final}")
print(f"Final Model - R^2 Score: {r2_final}")

rmse_final = mse_final ** 0.5
mae_final = mean_absolute_error(y_test, y_pred_final)

print(f"Final Model - Root Mean Squared Error (RMSE): {rmse_final}")
print(f"Final Model - Mean Absolute Error (MAE): {mae_final}")

Fitting 3 folds for each of 108 candidates, totalling 324 fits
Best Parameters from GridSearchCV: {'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}
Final Model - Mean Squared Error: 0.0003010796193983277
Final Model - R^2 Score: 0.9999998391483166
Final Model - Root Mean Squared Error (RMSE): 0.01735164601409122
Final Model - Mean Absolute Error (MAE): 0.002372827988054425
```

D3:CODE EXECUTION

Please modify the local file path

Local_path = 'C:/Users/beatr/OneDrive/Desktop/D209/Task2/'

```
#File path variables

Local_path = 'C:/Users/beatr/OneDrive/Desktop/D209/Task2/'

InputFilePath = Local_path + 'churn_clean.csv'

OutputfilePath = Local_path + 'churn_cleaned_df.csv'

XtrainOutputfilePath =Local_path + 'Xtrain.csv'

XtestOutputfilePath =Local_path + 'Xtest.csv'

ytrainOutputfilePath =Local_path + 'ytrain.csv'

ytestOutputfilePath =Local_path + 'ytest.csv'

TargetVariable = "MonthlyCharge"
```



BeatriceMwangi_D2
09_DataMining1_Ta:

E1: ACCURACY AND MSE

1. Accuracy

Accuracy refers to how well the model predicts or classifies data.

In the Random Forest regressor model, the following Metrics were used to evaluate the accuracy R^2 score, RMSE, and MAE

The R^2 score is extremely close to 1, which suggests that the final model explains 99.99998% of the variance in the target variable(MonthlyCharge). This means the model has a near-perfect fit to the data and accurately predicts the target values.

The RMSE measures the average magnitude of errors in the model's predictions. A smaller RMSE is better, and the final model has an RMSE of 0.017351646, indicating that, on average, the model's predictions deviate from the actual values by only about 0.017 units. This small value suggests that the model's predictions are highly accurate, with little deviation from the true values.

The MAE represents the average absolute difference between the predicted and actual values. A lower MAE indicates better accuracy. With a value of 0.002372828, this means that, on average, the model's predictions are off by only 0.00237 units. This small value further highlights the model's high accuracy and ability to make predictions that are very close to the actual values.

Based on the accuracy Metrics, the model can make predictions accurately.

2. Mean Squared Error (MSE):

The Mean Squared Error (MSE) measures the average squared difference between the predicted and actual values. A lower MSE indicates that the model's predictions are closer to the true

values, which suggests better performance. The MSE of 0.0003010796 measures the average squared difference between predicted and actual values. A low MSE value indicates better Random Forest Regressor model performance. The MSE is quite low in this case, reinforcing that the model minimizes prediction errors.

E2: RESULTS AND IMPLICATIONS

The final Random Forest model demonstrates outstanding performance with a nearly perfect R^2 score of 0.999998391 and a very low Mean Squared Error (MSE) of 0.0003010796. These metrics suggest that the model captures almost all of the variance in the target variable, Monthly Charge, making it an extremely reliable tool for prediction. The low Root Mean Squared Error (RMSE) of 0.017351646 and Mean Absolute Error (MAE) of 0.002372828 further confirm the model's accuracy, with minimal deviation from the actual values. However, the exceptionally high R^2 score could indicate a potential for overfitting, where the model might perform well on the training data but may not generalize as effectively to unseen data.

E3: LIMITATION

The analysis was limited due to a lack of understanding of the business data, how the data was collected, and the domain knowledge.

E4: COURSE OF ACTION

I recommend further data analysis and potential validation on a separate test to ensure the model's robustness with unseen data. Also, given the model's impressive performance, I recommend that the telecommunications industry consider implementing this random forest regression model to predict Monthly Charges with high accuracy.

F: PANOPTO RECORDING

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=882010f0-fc3c-479b-88c7-b272000bc52e>

G: SOURCES FOR THIRD-PARTY CODE

None

H: SOURCES

Breiman, Leo (2001, October), Random Forests

<https://link.springer.com/article/10.1023/A:1010933404324>

Saxena, Shipra (2024, September 18) What are Categorical Data Encoding Methods | Binary Encoding

<https://www.analyticsvidhya.com/blog/2020/08/types-of-categorical-data-encoding/>

Charles, Chibuike (2023, June 2) Random Forest; Assumptions, Advantages, Disadvantages and Applications.

<https://medium.com/@chibuike.odo.c/random-forest-assumptions-advantages-disadvantages-and-applications-2881f4ea14b6#:~:text=There%20should%20be%20actual%20values,random%20guess%20by%20the%20algorithm.&text=The%20data%20is%20normally%20distributed.&text=The%20prediction%20from%20the%20individual%20trees%20must%20have%20very%20low%20correlations.&text=The%20input%20data%20is%20continuous%20and%20the%20target%20data%20is%20discrete.>

I: PROFESSIONAL COMMUNICATION